



Containers, images, and building your own

Salvo Nicotra

From source code to runtime environments

“it works on my machine”

A program depends on more than its source code: runtime, libraries, system packages, configuration, and assumptions about the environment include the operating system, file system, architecture, and network.

Examples:

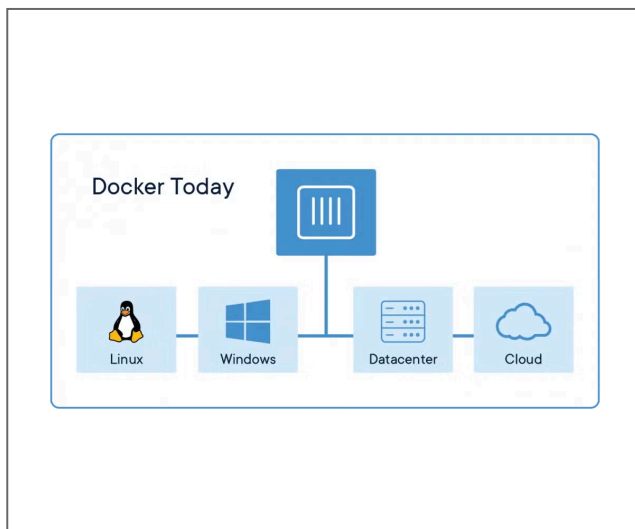
- Java depends on JVM, libraries, and system packages.
- Python depends on the Python interpreter, standard library, and third-party packages.
- PHP depends on the PHP interpreter, extensions, and system packages.
- ...

Note

How can we package and run the same application on a laptop, a server, or a cloud platform?

What is a Container?

- A standardized unit of software
- A container is a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another



Containers 101

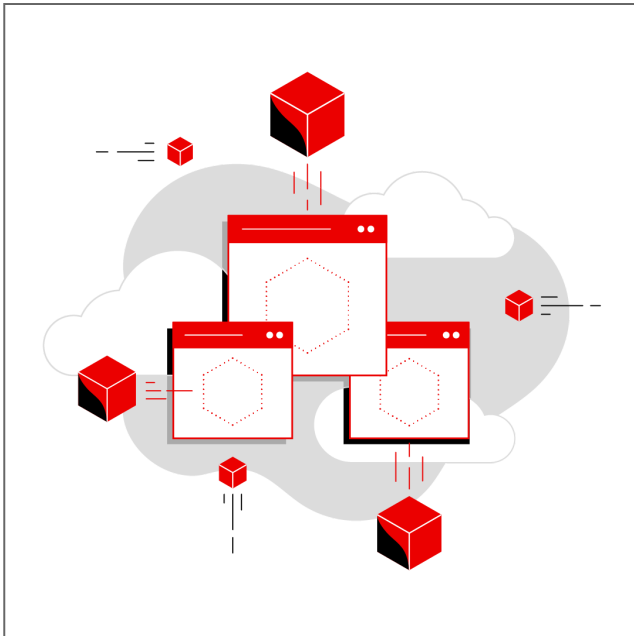
- Containers are packages of software that contain all of the necessary elements to run in any environment.
- In this way, containers virtualize the operating system and run anywhere, from a private data center to the public cloud or even on a developer's personal laptop.
- From Gmail to YouTube to Search, everything at Google runs in containers.
- Containerization allows our development teams to move fast, deploy software efficiently, and operate at an unprecedented scale



NicsMeme

What's a Linux container?

- A Linux® container is a set of one or more processes that are isolated from the rest of the system.
- All the files necessary to run them are provided from a distinct image, meaning that Linux containers are portable and consistent as they move from development, to testing, and finally to production.
- This makes them much quicker than development pipelines that rely on replicating traditional testing environments.
- Because of their popularity and ease of use containers are also an important part of IT security.



Container vs Virtual Machines

CONTAINER

Containers are an abstraction at the app layer that packages code and dependencies together. Multiple containers can run on the same machine and share the OS kernel with other containers, each running as isolated processes in user space. Containers take up less space than VMs (container images are typically tens of MBs in size), can handle more applications and require fewer VMs and Operating systems.

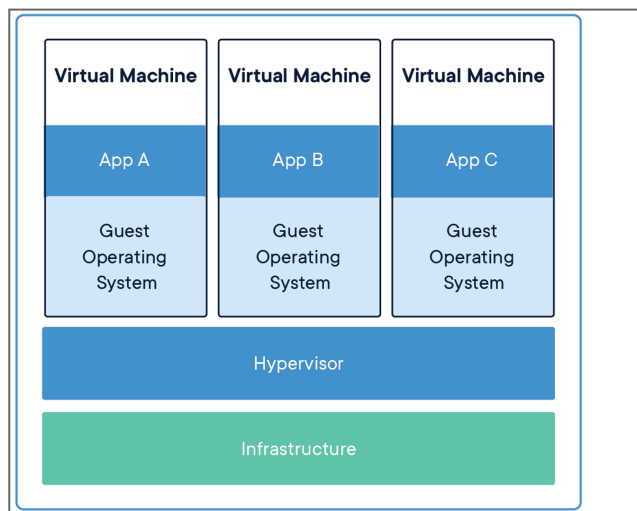
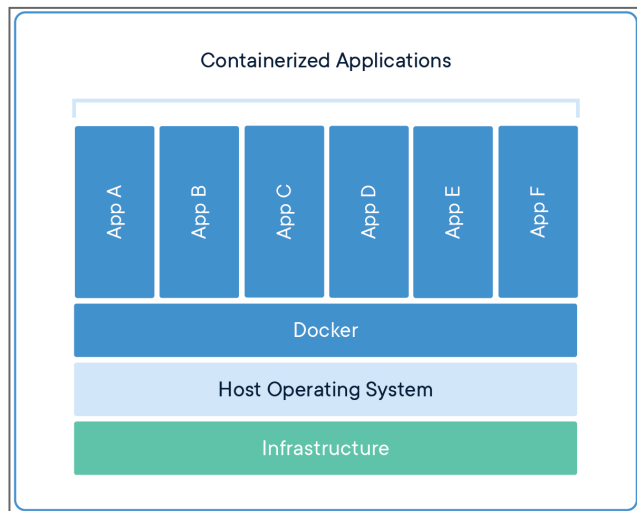
VIRTUAL MACHINES

Virtual machines (VMs) are an abstraction of physical hardware turning one server into many servers. The hypervisor allows multiple VMs to run on a single machine.

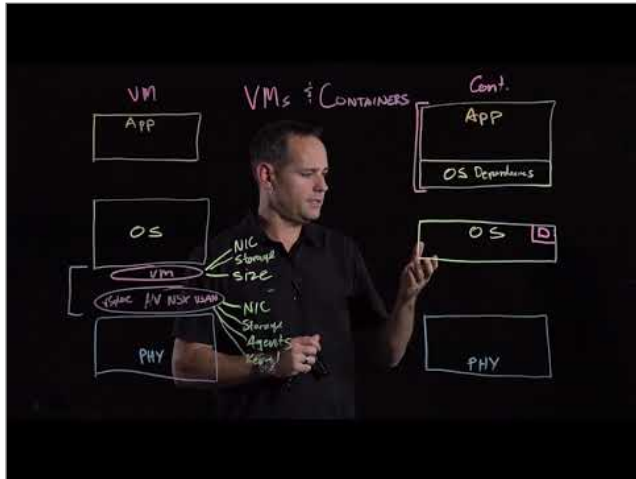
Each VM includes a full copy of an operating system, the application, necessary

binaries and libraries - taking up tens of GBs. VMs can also be slow to boot

Visual comparison containers vs virtual machines

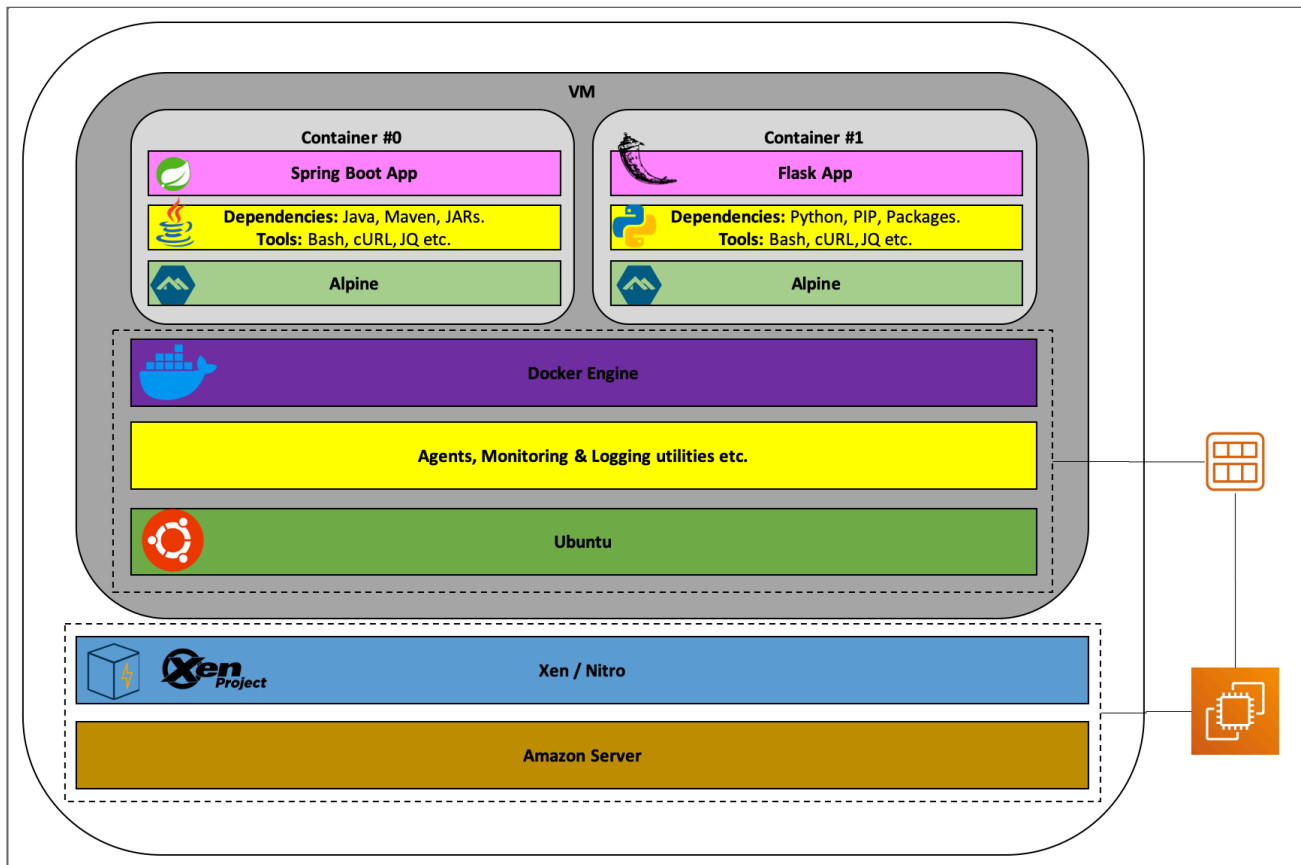


Some Videos



Containers & Virtual Machines

<https://medium.com/better-programming/containers-and-virtual-machines-an-overview-94c558905b6>



Open Container Initiative (OCI)

Docker made containers accessible, but portability depends on shared formats and interfaces—not on every machine running the same vendor product.

The most important contributions of Docker to the container ecosystem is the Open Container Initiative (OCI), which defines open standards for container formats and runtimes.

- **Image Specification:** standardizes image configuration, manifests, and filesystem layers.
- **Runtime Specification:** defines the configuration and lifecycle of a container runtime bundle.
- **Distribution Specification:** describes how images are pushed to and pulled from registries.

Docker

A Docker container image is a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries and settings.

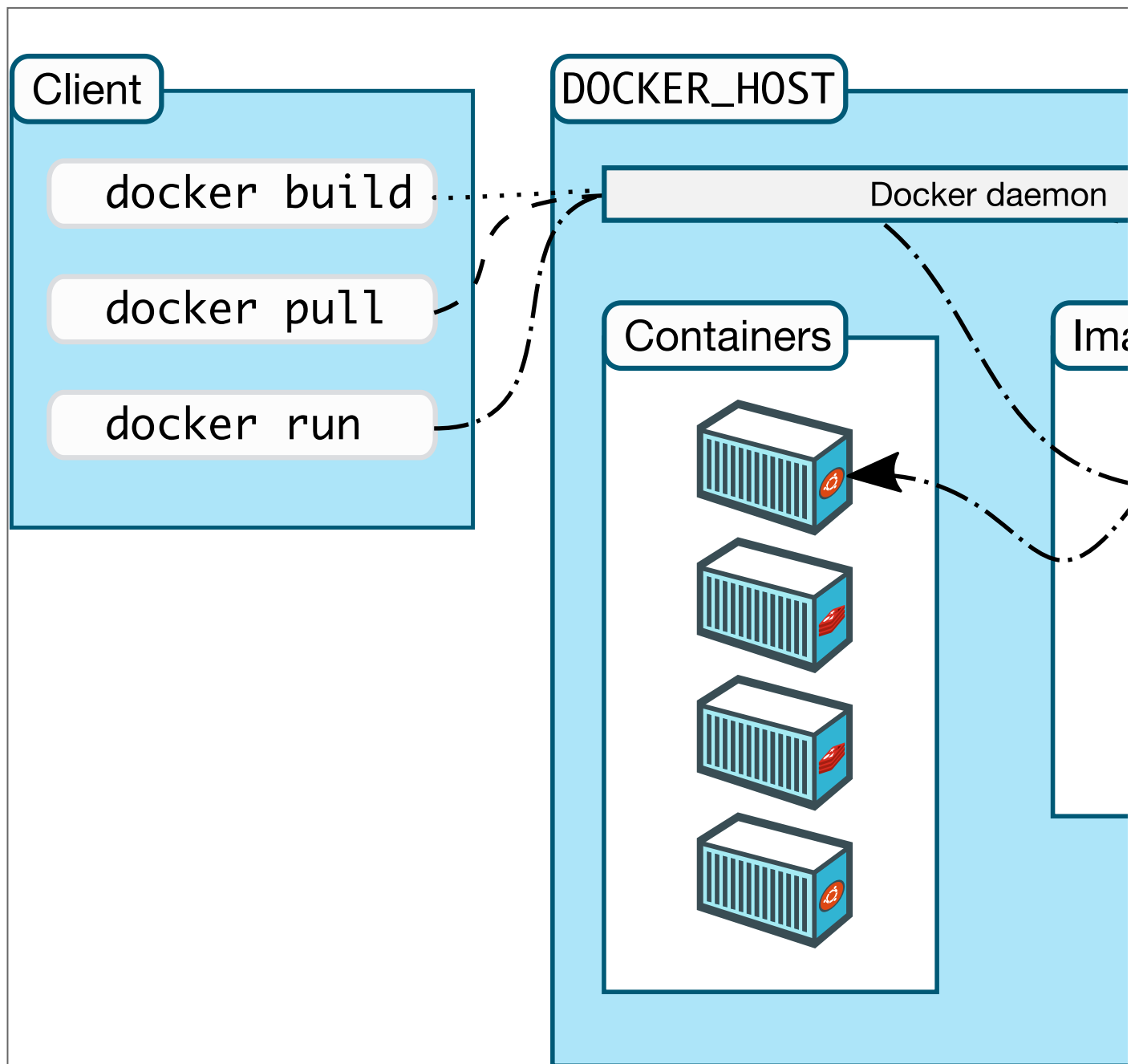
In Docker **images** become **containers** when they run on Docker Engine.

Features

Docker containers that run on Docker Engine:

- **Standard:** Docker created the industry standard for containers, so they could be portable anywhere
- **Lightweight:** Containers share the machine's OS system kernel and therefore do not require an OS per application, driving higher server efficiencies and reducing server and licensing costs
- **Secure:** Applications are safer in containers and Docker provides the strongest default isolation capabilities in the industry

Docker Architecture



Set up a local Docker environment

[See docs](#)



Windows (WSL)

HISTORY FROM DOCKER BLOG

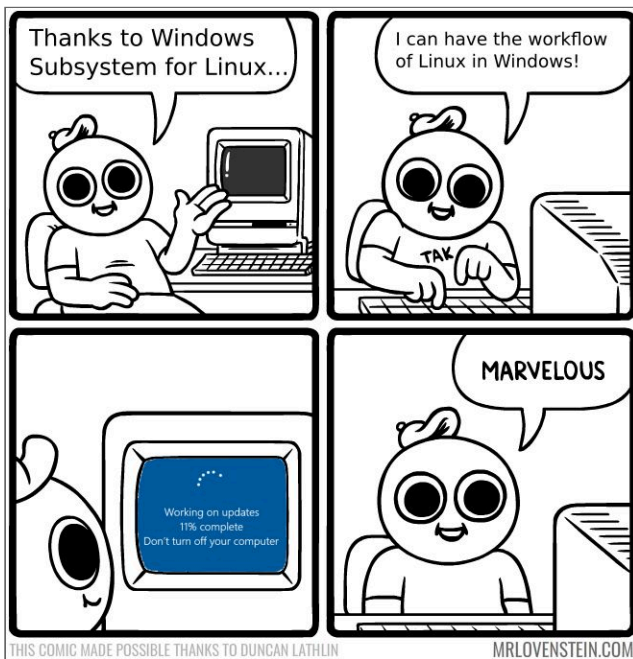
Date	Title
Nov 22 2024	Docker Desktop 4.36: New Enterprise Administration Features, WSL 2, and ECI Enhancements
Jul 11 2023	<u>Optimizing Deep Learning Workflows: Leveraging Stable Diffusion and Docker on WSL 2</u>
Dec 15 2021	<u>WSL 2 GPU Support for Docker Desktop on NVIDIA GPUs</u>
Dec 21 2020	<u>WSL 2 GPU Support is Here</u>
Aug 20 2020	<u>Docker Desktop & WSL 2 – Backport Update</u>
May 20 2020	<u>Creating the best Linux Development experience on Windows & WSL 2</u>
May 04 2020	<u>Docker Desktop: WSL 2 Best practices</u>
Apr 13 2020	<u>How we test Docker Desktop with WSL 2</u>
Jun 16 2019	<u>Docker ❤️ WSL 2 – The Future of Docker Desktop for Windows</u>
Jul 31 2019	<u>5 Things to Try with Docker Desktop WSL 2 Tech Preview - Link Broken</u>

Date	Title
Aug 14 2019	<u>Developing Docker-Powered Apps on Windows with WSL</u>
Oct 24 2019	<u>Introducing the Docker Desktop WSL 2 Backend</u>

From Docker Docs

- Windows Subsystem for Linux (WSL) 2 is a full Linux kernel built by Microsoft, which lets Linux distributions run without managing virtual machines. With Docker Desktop running on WSL 2, users can leverage Linux workspaces and avoid maintaining both Linux and Windows build scripts. In addition, WSL 2 provides improvements to file system sharing and boot time.
- Docker Desktop uses the dynamic memory allocation feature in WSL 2 to improve the resource consumption. This means Docker Desktop only uses the required amount of CPU and memory resources it needs, while allowing CPU and memory-intensive tasks such as building a container, to run much faster.
- Additionally, with WSL 2, the time required to start a Docker

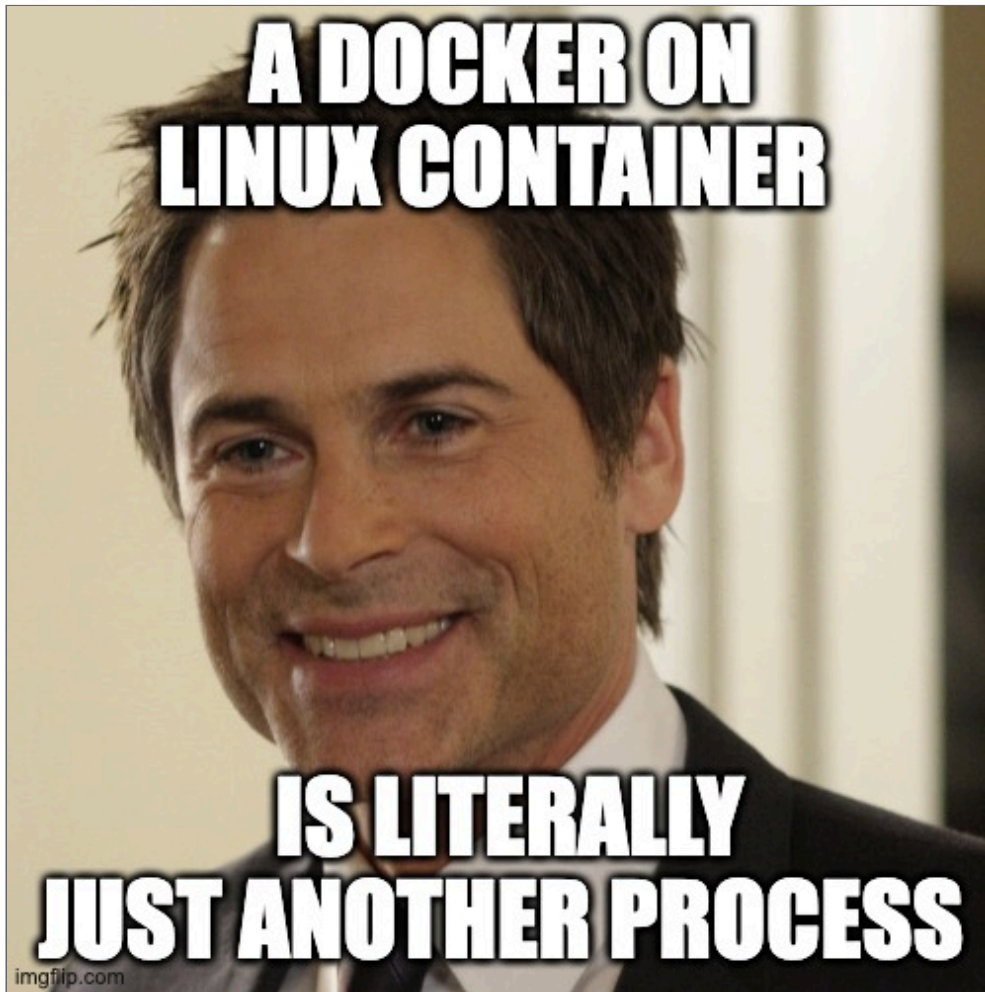
daemon after a cold start is
significantly faster



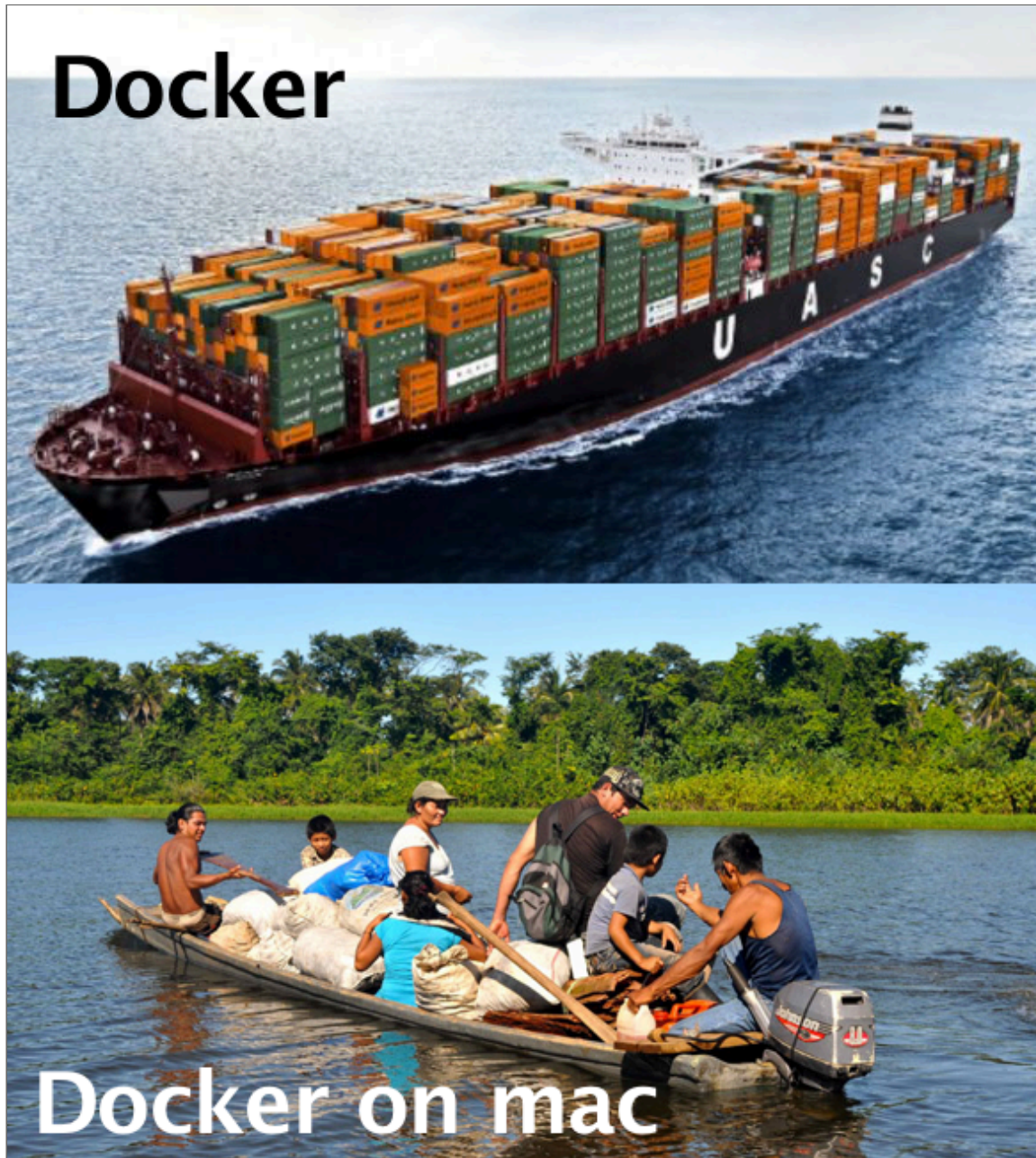
)

<https://devrant.com/rants/1933665/marvelous>

Linux



Mac



<https://www.reddit.com/r/ProgrammerHumor/comments/d0ckĩ>

Docker Desktop for Apple silicon

Unlike Intel-based Macs,
Apple silicon Macs cannot run
x86-64 binaries natively.

Docker Desktop for Apple
silicon uses a combination of
emulation and virtualization
to run x86-64 containers on
Apple silicon using Rosetta 2
and QEMU.

The first release in 2021

started to supported Apple
silicon Macs

However Docker and Mac
relationship is not always
easy...



Let's start using Docker

Docker version

```
1 # Docker version
2 docker --version
3 Docker version 29.5.2, build 79eb04c
```

Hello World

```
1 $ docker run hello-world
2 Unable to find image 'hello-world:latest' locally
3 latest: Pulling from library/hello-world
4 4f55086f7dd0: Pull complete
5 d5e71e642bf5: Download complete
6 Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
7 Status: Downloaded newer image for hello-world:latest
8
9 Hello from Docker!
10 This message shows that your installation appears to be working correctly.
11
12 To generate this message, Docker took the following steps:
13 1. The Docker client contacted the Docker daemon.
14 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
15    (amd64)
16 3. The Docker daemon created a new container from that image which runs the
17    executable that produces the output you are currently reading.
18 4. The Docker daemon streamed that output to the Docker client, which sent it
19    to your terminal.
20
21 To try something more ambitious, you can run an Ubuntu container with:
22 $ docker run -it ubuntu bash
23
24 Share images, automate workflows, and more with a free Docker ID:
25 https://hub.docker.com/
26
27 For more examples and ideas, visit:
28 https://docs.docker.com/get-started/
```

List Images

```
1 docker image ls
```

```
2
```

```
3 IMAGE          ID          DISK USAGE  CONTENT SIZE  EXTRA
```

```
4 hello-world:latest 5e2309035332 25.9kB      9.49kB       U
```

[i Info](#) → [U In Use](#)

List Containers

```
1 $ docker container ls -a
2 CONTAINER ID   IMAGE      COMMAND                  CREATED          STATUS          PORTS          NAMES
3 5bfd23731b3b   hello-world  "/hello"                About an hour ago  Exited (0) About an hour ago          elegant_lewin
```

Docker Container Name

name defined Assign name (–name) The –name flag lets you specify a custom identifier for a container. The following example runs a container named test using the nginx:alpine image in detached mode.

```
1 docker run --name test -d nginx:alpine
2 4bed76d3ad428b889c56c1ecc2bf2ed95cb08256db22dc5ef58
3
4 docker ps
5 CONTAINER ID   IMAGE     COMMAND
6 4bed76d3ad42   nginx:alpine  "/docker-entrypoint...."
```

You can reference the container by name with other commands. For example, the following commands stop and remove a container named test:

```
1 docker stop test
2 test
3 docker rm test
4 test
```

name not defined Name it's random defined by a fancy algorithm names-generator.go

Moreover, if you connect the container to a user-defined

bridge network, other containers on the same network can refer to the container by name via DNS.

Ubuntu

```
1 docker run -it ubuntu
```

Docker Container Lifecycle

- What image and command created this container? `docker inspect`
- What did the application print? `docker logs`
- Is it running, and which ports are published? `docker ps`
- Can I inspect a live process? `docker exec -it <container> sh`

A container's writable layer is disposable. Use a named volume when data must survive container removal, for example `docker run -v app-data:/data ....`

A container is only one part of an application

A web application often needs more than a process:

- **Ports** connect clients to the service.
- **Volumes** keep data outside the container's writable layer.
- **Networks** let related containers communicate.

Docker Compose

Compose describes a local multi-container application in a `compose.yaml` file and starts its services together.

```
1 services:
2   web:
3     build: .
4     ports: ["8080:8080"]
5     depends_on: [cache]
6   cache:
7     image: redis:7-alpine
```

```
1 docker compose up --build
2 docker compose down
```

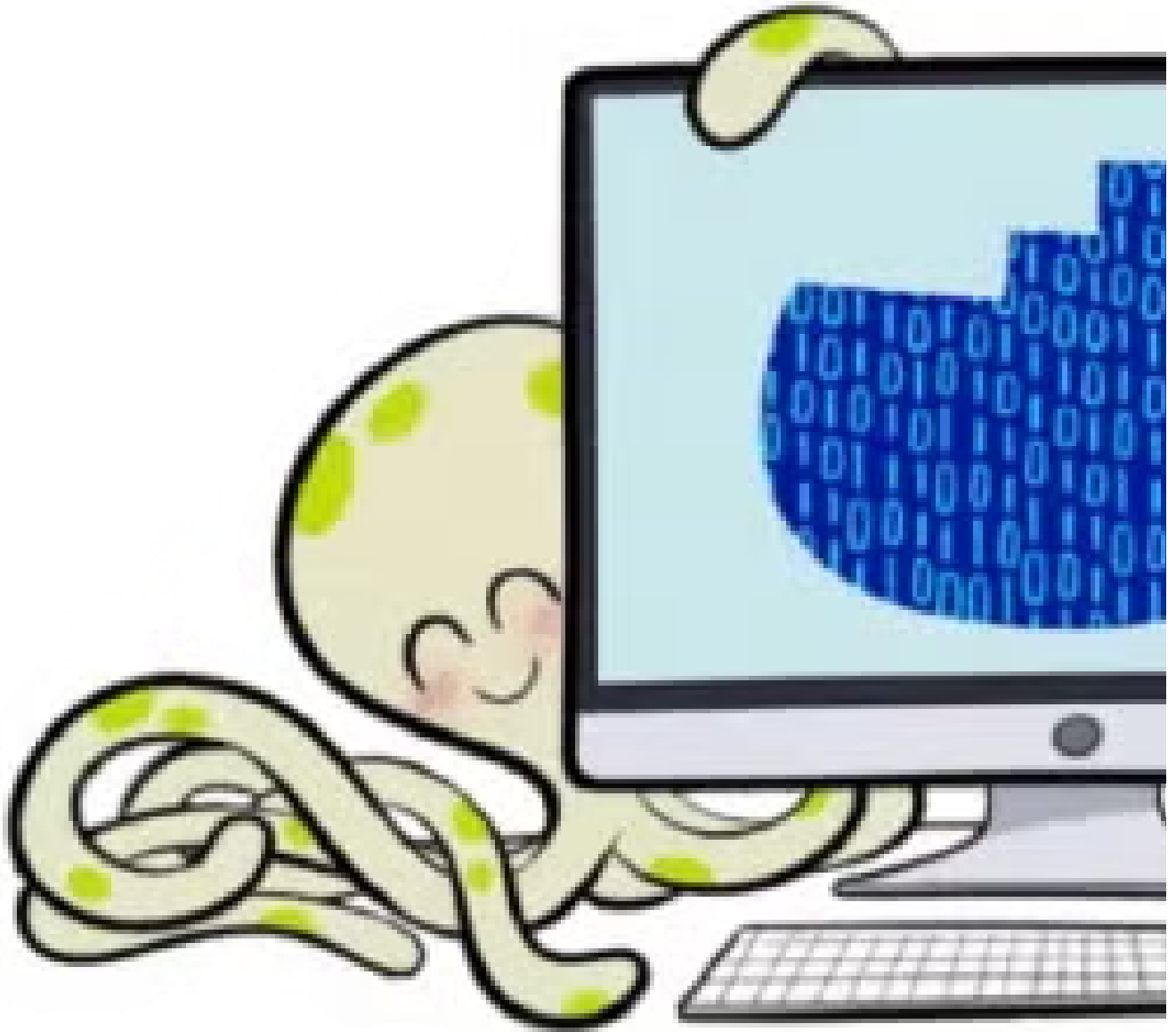
Compose is useful for development and small deployments; it is not a cluster scheduler like Kubernetes. `depends_on` controls startup ordering, but applications may still need to retry until dependencies are ready.

Play with Docker

Docker playground

A simple, interactive and fun playground to learn Docker

<https://www.docker.com/play-with-docker>



Docker Components

Images

Tap Quotes

- An image is a read-only template with instructions for creating a Docker container.
- Often, an image is based on another image, with some additional customization.
- For example, you may build an image which is based on the ubuntu image, but installs the Apache web server and your application, as well as the configuration details needed to make your application run.

Image layers

- You might create your own images or you might only use those created by others and published in a registry.
- To build your own image, you create a Dockerfile with a simple syntax for defining the steps needed to create the image and run it.
- Each instruction in a Dockerfile creates a layer in the image. When you change the Dockerfile and rebuild the image, only those layers which have changed are rebuilt.
- This is part of what makes images so lightweight, small, and fast, when compared to other virtualization technologies.

<https://ragin.medium.com/docker-what-it-is-how-images-are-structured-docker-vs-vm-and->

some-tips-part-1-
d9686303590f

Containers

A container is a runnable instance of an image. You can create, start, stop, move, or delete a container using the Docker API or CLI. You can connect a container to one or more networks, attach storage to it, or even create a new image based on its current state.

By default, a container is relatively well isolated from other containers and its host machine. You can control how isolated a container's network, storage, or other underlying subsystems are from other containers or from the host machine.

A container is defined by its image as well as any configuration options you provide to it when you create or start it. When a container is removed, any changes to its state that are not stored in persistent storage disappear.

Docker Hub

Docker Hub is a service provided by Docker for finding and sharing container images with your team. It is the world's largest repository of container images with an array of content sources including container community developers, open source projects and independent software vendors (ISV) building and distributing their code in containers.

<https://docs.docker.com/docker-hub/>

For production environments consider rate limits set in November 2020

Docker CLI

The `docker run` command first **creates** a writeable container layer over the specified image, and then **starts** it using the specified command.

That is, `docker run` is equivalent to the API

`/containers/create` then `/containers/(id)/start`.

A stopped container can be restarted with all its previous changes intact using `docker start`.

See `docker ps -a` to view a list of all containers.

The `docker run` command can be used in combination with `docker commit` to change the command that a container runs.

<https://docs.docker.com/engine/reference/commandline/run/>

Visual Studio Code Integration

The Docker extension makes it easy to build, manage, and deploy containerized applications from Visual Studio Code. This page provides an overview of the Docker extension capabilities; use the side menu to learn more about topics of interest. If you are just getting started with Docker development, try the Docker tutorial first to understand key Docker concepts

<https://code.visualstudio.com/docs/containers/overview>

Build Your Own Container

Build an image using Dockerfile

NicsMeme inspired by Docker Anti Patterns

Let's start with an example

Node Bulletin

Clone the sample and inspect its application and Dockerfile:

```
1 git clone https://github.com/dockeramples/node-bulletin-board
2 cd node-bulletin-board/bulletin-board-app
```

A Dockerfile describes a repeatable build. This example separates dependency files from frequently changing application source so the dependency layer can be reused:

```
1 FROM node:current-slim
2 WORKDIR /usr/src/app
3 COPY package*.json ./
4 RUN npm install
5 COPY . .
6 EXPOSE 8080
7 CMD ["npm", "start"]
```

For a reproducible production build, pin the base image deliberately and use the project's committed lockfile (with `npm ci` when applicable).

```
1 docker image build -t bulletinboard:1.0 .
2 docker container run -d -p 8080:8080 --name myapp bulletinboard:1.0
```

Check the running application

```
1 docker ps
2 docker logs myapp
3 docker inspect myapp
```

Open <http://localhost:8080>. In `-p 8080:8080`, the first port is on the host and the second is inside the container. [EXPOSE](#) documents the intended container port; it does not publish it.

```
1 docker stop myapp
2 docker rm myapp
```

Try your self

Go to [Docker Sample](#)

Choose an example and try to run it

Before building: the build context

In `docker build -t bulletinboard:1.0 .`, the final `.` is the **build context**: the files available to the builder. A large or careless context slows builds and may expose files unintentionally.

Add a `.dockerignore` file at the context root, for example:

```
1 .git
2 node_modules
3 .env
4 *.log
```

Do not copy credentials into an image. Excluding `.env` is not a substitute for proper runtime or BuildKit secret handling.

Build command

<https://docs.docker.com/build/>

Run the cli command `docker build`

- without arguments search for a Dockerfile in the same directory
- -f option specify the path of the docker file `bash $ docker build -f /path/to/a/Dockerfile`
- -t or --tag define a name for the image

A Dockerfile is also a build plan

Layer order affects both rebuild time and image contents:

1. Copy dependency manifests first.
2. Install dependencies in a stable layer.
3. Copy frequently changing source afterward.

If only source code changes, the dependency installation layer can be reused. BuildKit improves the build experience with features such as cache mounts and secret mounts; secrets should not be baked into an image or passed as ordinary build arguments.

Reference: Docker build cache and Build secrets.

Syntax

<https://docs.docker.com/reference/dockerfile/>

- Docker can build images automatically by reading the instructions from a Dockerfile.
- A Dockerfile is a text document that contains all the commands a user could call on the command line to assemble an image.

```
1 # Comment
2 INSTRUCTION arguments
```

Convention: instructions are in UPPER case

FROM

<https://docs.docker.com/engine/reference/builder/#from>

```
1 FROM [--platform=<platform>] <image>[:<tag>] [AS <name>]
```

- First instruction after comments/parser directives/arg
- Indicates the Image source (from a registry)
- Can use ARG parameter
- it's your baby!

ARG

<https://docs.docker.com/engine/reference/builder/#arg>

Define environment variable used in the Builder

```
1 ARG <name>[=<default value>]
```

Argument can be used in DockerFile FROM, RUN, ... also passed during the BUILD

```
1 $ docker build --build-arg user=what_user
```

ARG are referenced using standard

`${VARIABLE}` notation

Be careful with ARG

- scope starts from the line where is defined
- ENV wins on ARG in RUN
- There are a set of predefined ARG (like HTTP_PROXY)
- Don't pass secret in ARG

ENV

<https://docs.docker.com/engine/reference/builder/#env>

Define enviroment variable
that *persist* in the container

```
1 ENV <key> <value>
2 ENV <key>=<value> ...
```

Enviroment variable can be
also passed while starting

```
1 $ docker run -e "deep=purple"
```

RUN (shell mode)

<https://docs.docker.com/engine/reference/builder/#env>

Execute commands in the *layer* on top of the current image

(shell form, the command is run in a shell, which by default is `/bin/sh -c` on Linux or `cmd /S /C` on Windows)

```
1 RUN <command>
```

The default shell for the shell form can be changed using the `SHELL` command.

In the shell form you can use a (backslash) to continue a single `RUN` instruction onto the next line. For example, consider these two lines:

```
1 RUN /bin/bash -c 'source $HOME/.bashrc; \  
2 echo $HOME'
```

RUN (exec mode)

<https://docs.docker.com/engine/reference/builder/#run>

```
1 RUN ["executable", "param1", "param2"] (exec form)
```

Unlike the shell form, the exec form does not invoke a command shell. This means that normal shell processing does not happen.

To use a different shell, other than `/bin/sh`, use the exec form passing in the desired shell. For example,

```
1 RUN ["/bin/bash", "-c", "echo hello"]
```

For example, `RUN [ "echo", "$HOME" ]` will not do variable substitution on `$HOME`. If you want shell processing then either use the shell form or execute a shell directly, for example:

```
1 RUN [ "sh", "-c", "echo $HOME" ].
```

The exec form is parsed as a JSON array, which means that you must use double-quotes (") around words not single-quotes (').

Confused ?

- RUN actually runs a command and commits the result
- CMD does not execute anything at build time, but specifies the intended command for the image.

CMD (exec form)

The main purpose of a CMD is to provide defaults for an executing container. These defaults can include an executable, or they can omit the executable, in which case you must specify an ENTRYPOINT instruction as well.

Exec form, this is the preferred form

```
1 CMD ["executable","param1","param2"]
```

Like RUN exec form do not invoke a shell, so in order to force the variable substitution done by shell use

```
1 CMD [ "sh", "-c", "echo $HOME" ]
```

<https://docs.docker.com/engine/reference/builder/#env>

CMD (shell form)

The main purpose of a CMD is to provide defaults for an executing container. These defaults can include an executable, or they can omit the executable, in which case you must specify an ENTRYPOINT instruction as well.

```
1 CMD command param1 param2
```

If you want to run your without a shell then you must express the command as a JSON array and give the full path to the executable. This array form is the preferred format of CMD. Any additional parameters must be individually expressed as strings in the array:

```
1 FROM ubuntu
2 CMD echo "This is a test." | wc -l
```

<https://docs.docker.com/engine/reference/builder/#cmd>

ENTRYPOINT

<https://docs.docker.com/reference/dockerfile/#entrypoint>

Note

An **ENTRYPOINT** allows you to configure a container that will run as an executable.

- Exec form **DockerFile**
`ENTRYPOINT ["executable", "param1", "param2"]`
- Shell form **DockerFile**
`ENTRYPOINT command param1 param2`

Example

- Run a container from **Official Image**
`bash docker run -i -t --rm -p 80:80 nginx`
- Test that it works correctly on the machine
- Try `bash docker run -i -t -rm -p 80:80 nginx cat docker-entrypoint.sh`

- Let give a look to the Dockerfile

CMD - ENTRYPOINT interaction

<https://docs.docker.com/reference/dockerfile/#understand-how-arg-and-from-interact>

Both CMD and ENTRYPOINT instructions define what command gets executed when running a container. There are few rules that describe their co-operation.

- Dockerfile should specify at least one of CMD or ENTRYPOINT commands.
- ENTRYPOINT should be defined when using the container as an executable.
- CMD should be used as a way of defining default arguments for an ENTRYPOINT command or for executing an ad-hoc command in a container

Additional Dockerfile instructions

The core build and startup flow is in place. These instructions describe supporting runtime behavior and file handling.

WORKDIR

<https://docs.docker.com/reference/dockerfile/#workdir>

```
1 WORKDIR /path/to/workdir
```

- The WORKDIR instruction sets the working directory for any RUN, CMD, ENTRYPOINT, COPY and ADD instructions that follow it in the Dockerfile.
- If the WORKDIR doesn't exist, it will be created even if it's not used in any subsequent Dockerfile instruction.
- Can be used multiple times

```
1 WORKDIR /a
2 WORKDIR b
3 WORKDIR c
4 RUN pwd
```

The output of the final pwd command in this Dockerfile would be /a/b/c.

COPY

The COPY instruction copies new files or directories from and adds them to the filesystem of the container at the path .

```
1 COPY [--chown=<user>:<group>] <src>... <dest>  
2 COPY [--chown=<user>:<group>] ["<src>",... "<dest>"] (this form is required for paths containing whitespace)
```

COPY and **ADD**: prefer the explicit operation

Use **COPY** for ordinary files and directories from the build context:

```
1 COPY package*.json ./
2 COPY . .
```

ADD has additional behaviors (such as automatic extraction of local tar archives and remote URL sources). Use it when those behaviors are intended; otherwise **COPY** is clearer.

References: COPY, ADD.

EXPOSE

<https://docs.docker.com/reference/dockerfile/#expose>

The EXPOSE instruction informs Docker that the container listens on the specified network ports at runtime.

```
1 EXPOSE <port> [<port>/<protocol>...]
```

- The EXPOSE instruction does not actually publish the port.
- It functions as a type of documentation between the person who builds the image and the person who runs the container, about which ports are intended to be published.
- To actually publish the port when running the container, use the -p flag on docker run to publish and map one or more ports, or the -P flag to publish all exposed ports and map them to high-order ports.
- `bash docker run -p 80:80/tcp -p 80:80/udp`

VOLUME: declare a data mount point

VOLUME `/data` marks a path intended for externally managed storage. It does not by itself choose a host directory or make the application's data durable in every deployment. For a local named volume, mount it when running the container:

```
1 docker run -v app-data:/data my-image
```

The volume lifecycle is separate from the container lifecycle; inspect and back it up according to the application's needs.

A few useful instructions beyond the core

- **USER** selects the user for subsequent build steps and the default runtime process. Prefer a non-root application user when practical.
- **HEALTHCHECK** lets Docker report whether a container is healthy; it does not replace application-level monitoring.
- **LABEL** adds image metadata such as maintainer or source information.

These are worth recognizing, but they are not a checklist to add blindly to every Dockerfile.

References: [Dockerfile reference](#) and [Docker build best practices](#).

Best Practices

https://docs.docker.com/develop/develop-images/dockerfile_best-practices/

Multistage

<https://docs.docker.com/build/building/multi-stage/>

- With multi-stage builds, you use multiple FROM statements in your Dockerfile.
- Each FROM instruction can use a different base, and each of them begins a new stage of the build.
- You can selectively copy artifacts from one stage to another, leaving behind everything you don't want in the final image

Multistage Examples

Java

```
1 FROM maven:alpine as build
2 ENV HOME=/usr/app
3 RUN mkdir -p $HOME
4 WORKDIR $HOME
5 ADD . $HOME
6 RUN mvn package
7
8 FROM openjdk:8-jdk-alpine
9 COPY --from=build /usr/app/target/single-module-cac
10 ENTRYPOINT java -jar /app/runner.jar
```

Python

```
1 # Build Stage : [ Step 1 - 3 ]
2 FROM python:3.10-slim AS build
3
4 # Install Dependencies
5 RUN apt-get update && apt-get -y install libpq-dev
6 COPY ./requirements.txt requirements.txt
7 RUN pip3 install --no-cache-dir --target=packages -
8
9 # Runtime Stage [ Step 4 - 5 ]
10 FROM python:3.10-slim AS runtime
11 # Copying Dependencies from build stage
12 COPY --from=build packages /usr/lib/python3.10/site
13 ENV PYTHONPATH=/usr/lib/python3.10/site-packages
14
15 # Security Context
16 RUN useradd -m nonroot
17 USER nonroot
18
19 # Env configuration [ Step 6 ]
20 WORKDIR /app
21 COPY . .
22 EXPOSE 8080
23 ENTRYPOINT ["python", "app.py"]
```

To commit or not to commit?

<https://medium.com/@lizrice/to-commit-or-not-to-commit-5ab72f9a466e>

Don't do it.

No, really, I know it's tempting, but if you're ever going to use this container image again, don't use commit.

There are significant downsides to the commit approach:

- You can't reproduce the image
- You can't change the base image

Creating a Dockerfile might feel a bit like work, but if you might want this image again, it's the approach you need to take.

Il cannolo della fortuna

A simple fortune extended with sicilian proverbs

Design

Sources

- https://it.wikiquote.org/wiki/Proverbi_siciliani

Build

```
1 cd fortune
2 docker build -t tap:fortune .
```

Code

```
1 FROM ubuntu
2 RUN apt-get update && apt-get -y install fortunes fortunes fortune-mod
3 WORKDIR /usr/share/games/fortunes/
4 ADD sicilian .
5 RUN strfile -c % sicilian
6 ENTRYPOINT ["/usr/games/fortune"]
7 CMD ["sicilian"]
```

Demo

Beatu cui pri autru si castiga.

Breadth-first search is the bulldozer of science.
-- Randy Goebel

You have an image. Where should it run?

Once an application is packaged, the next questions are operational:

- Where is the image stored and distributed? **A registry.**
- Who schedules containers, restarts failures, and handles scaling?
- How much of the infrastructure do we want to manage ourselves?

Cloud offerings answer these questions at different levels. Compare their responsibilities rather than memorizing provider product names.

Running containers as a managed service

Amazon Web Services

Docker lets you build,
test, and deploy
applications quickly
<https://aws.amazon.com/docker/>

List of Docker AWS services
(updated Sep 2025)

Elastic Container Service (ECS).

Run highly secure,
reliable, and scalable
containers (2024)

Easily build, manage,
and run containerized
applications at any scale
(2025)

Amazon ECS is a fully managed container orchestration service that enables teams to build, manage, and run even the most demanding containerized workloads without the complexity of infrastructure management, freeing up development teams to innovate faster.



Elastic Kubernetes Service (EKS).

Build, run, and scale
production-ready
Kubernetes applications
easily across any
environment.

Amazon Elastic Kubernetes
Service (Amazon EKS) enables
teams of any size or skill level
to build, run, and scale
production-ready
applications easily across any
environment.



.

AWS Fargate

Serverless compute for containers

- AWS Fargate is a **serverless, pay-as-you-go** compute engine that lets you focus on building applications without managing servers.
- Moving tasks such as server management, resource allocation, and scaling to AWS does not only improve your operational posture, but also accelerates the process of going from idea to production on the cloud, and lowers the total cost of ownership.



AWS Batch

Batch processing for ML
model training,
simulation, and analysis
at any scale

AWS Batch is a fully managed batch computing service that plans, schedules, and runs your containerized batch ML, simulation, and analytics workloads across the full range of AWS compute offerings, such as Amazon ECS, Amazon EKS, AWS Fargate, and Spot or On-Demand Instances.



AWS Copilot

Command line interface
for containerized
applications

Azure

Wherever you are in your app modernization journey, accelerate your containerized application development while meeting your security requirements.

<https://azure.microsoft.com/en-us/product-categories/containers/>

Azure Kubernetes Service (AKS)

Deploy and scale
containers on managed
Kubernetes.

Azure Container Apps

Run AI apps and
microservices using
serverless containers.

App Services

Quickly build, deploy, and scale web apps and APIs on your terms. Work with .NET, .NET Core, Node.js, Java, Python or PHP, in containers or running on Windows or Linux. Meet rigorous, enterprise-grade performance, security and compliance requirements used a trusted, fully managed platform that handles over 40 billion requests per day.

Azure Container Instances

Launch containers with
hypervisor isolation.

Google Cloud

A better way to develop and deploy applications.

Everything at Google runs in containers Google has been using containers since the early 2000s—from Gmail to YouTube and Search, everything at Google operates in containers.

Since creating and open sourcing Kubernetes, an open source system to deploy, scale, and manage containerized applications anywhere, Google is committed to providing the easiest and safest way to manage Kubernetes.

Google Kubernetes Engine (GKE).

Kubernetes, evolved: The
foundation for platform
builders

Cloud Run

Build apps or websites
quickly on a fully
managed platform



.

Oracle Cloud

Oracle Cloud
Infrastructure (OCI)
Container Instances is a
serverless compute
service that enables you
to instantly run
containers without
managing any servers.



An experimental direction: Docker + WebAssembly

Docker + Wasm: an experimental integration (historical example)

Solomon Hykes, a co-founder of Docker, wrote in 2019, “If WASM+WASI existed in 2008, we wouldn’t have needed to create Docker. That’s how important it is. WebAssembly on the server is the future of computing.” Wasmer, out in version 1.0, provides “software containerization, we create universal binaries that work anywhere without modification, including operating systems like Linux, macOS, Windows, and web browsers. Wasm automatically sandboxes applications by default for secure execution”.

Wasm workloads

<https://docs.docker.com/desktop/features/wasm/>

Not only Docker

Docker Alternatives

1. Podman
2. Containerd
3. CRI-O
4. LXC
5. runc

Container technology - statistics & facts

Datanyze

Docker fund raising

As Docker gains momentum,
it hauls in \$105M Series C on
\$2B valuation

<https://techcrunch.com/2022/03/31/as-docker-gains-momentum-it-hauls-in-105m-series-c-on-2b-valuation/>

Still increasing

The market-share and revenue figures in this slide are a historical snapshot from the source material; refresh the data and date before using them as current statistics.

Speaker notes

Technologies for Advanced Programming (TAP) - 2026/2027